Métodos e Atributos

Dando dados e comportamentos aos seus objetos

Atributos

Atributos

O que são?

- Informações
- Características
- Dados que descrevem o objeto. Substantivos: nome, idade, saldo cor

Onde ficam?

- Cada objeto tem suas próprias informações
- Alterar um, não altera o outro

Atributos de instância

Os atributos de instância são informações que pertecem a cada objeto individualmente

São definidos no método especial “__init__”

Declarando atributos

```python
class Pessoa:
    def __init__(self, nome, idade):
        self.nome = nome
        self.idade = idade
```

Instanciando objetos com atributos

```
p1 = Pessoa("João", 30)
p2 = Pessoa("Maria", 25)
p3 = Pessoa("Pedro", 40)
```

Acessar atributos

- **objeto.atributo**

```
p1 = Pessoa("João", 30)

print(p1.nome)  # Output: João
print(p1.idade)  # Output: 30
```

Exemplo

```python
class Cachorro:
    def __init__(self, nome, raca, idade):
        # Cada linha abaixo cria um atributo no objeto
        self.nome  = nome
# recebe o valor passado na criação
        self.raca  = raca
        self.idade = idade

# Criando objetos e passando os valores dos atributos
rex  = Cachorro("Rex",  "Labrador", 3)
luna = Cachorro("Luna", "Poodle",   5)

# Acessando atributos com ponto
print(rex.nome)    # Rex
print(luna.idade)   # 5

# Atributos são independentes por objeto
rex.idade = 4       # só rex ficou mais velho
print(luna.idade)   # ainda 5
```

Dúvidas?

Métodos

Métodos

- Funções definidas dentro de uma classe.
- Representam os *comportamentos* do objeto - o que ele sabe fazer

Método X Função comum

Método	Função comum
Pertence a um objeto	Não pertence a nada
Opera sobre o objeto	Opera sobre dados soltos

Declarando métodos em uma classe

```
class Cachorro:

    def latir(self):
        print("Au au!")
```

O Self - referência ao próprio objeto

Na declaração dos métodos de uma classe, ele sempre precisa receber pelo menos um argumento, o *self*

Necessário para o python saber **sobre qual objeto operar**

Passado automaticamente na chamada do método, não sendo necessário passar ele explicitamente

Convenção Self

- o nome “self” é convenção, não obrigação

Tecnicamente você pode chamar esse parâmetro como quiser, funciona.

Mas não se deve fazer isso, pois a convenção “self” é universal na comunidade python

Dúvidas?

Acessando um método

- objeto.metodo()

```
c1 = Cachorro()
c1.latir()  # Output: Au au!
```

Substantivos X Verbos

Atributos são substantivos

- O cachorro tem **nome**
- A pessoa tem **idade**
- A conta bancária tem **saldo**

Métodos são verbos

- O cachorro pode **latir**
- a pessoa pode **falar**
- A conta bancária pode **sacar**

Método construtor

Construtor

- Método especial, executado **automaticamente** quando um objeto é criado
- Opcional

Serve para

- Inicializar dados do objeto
- Definir valores iniciais
- Deixar o objeto pronto para uso

Declarando o construtor (sintaxe)

```
class Pessoa:
    def __init__(self, nome, idade):
        self.nome = nome
        self.idade = idade
```

Valores padrão

```
class Pessoa:
    def __init__(self, nome, idade=0):
        self.nome = nome
        self.idade = idade

p1 = Pessoa("João", 30)
p2 = Pessoa("Maria")  # idade padrão 0
```

Recaptulando

Recapitulando

- Atributos
  - substantivos
- Métodos
  - verbos
- Construtor
  - Inicialização
- Self
  - Python usa para conhecer a referência do objeto

Exercícios